Vortex Runtime Decision Architecture

Gustavo Okamoto

18 August 2026

Publication metadata

- **Title:** Vortex Runtime Decision Architecture
- **Version:** 0.1
- **Author:** Gustavo Okamoto
- **Publication date:** 18 August 2026
- **DOI:** 10.5281/zenodo.21998321
- **Document license:** Creative Commons Attribution 4.0 International (CC BY 4.0)
- **Project:** Vortex
- **Source repository:** https://github.com/okamoto-security-labs/Vortex-DFS
- **Project website:** https://vortexdfs.com

Vortex Runtime Decision Architecture

Status: Draft **Version:** 0.1

1. Purpose

Vortex defines a runtime decision architecture for autonomous and agentic systems.

Its purpose is to determine whether a proposed action should be allowed to execute under the current runtime conditions.

Vortex treats execution as a decision problem with three independent inputs:

- Evidence: what supports the decision;
- Authority: what the actor is permitted to do;
- Consequence: what can happen if the action is wrong.

These inputs may influence policy together, but they are not interchangeable.

High-confidence evidence does not create authority.

Valid authority does not guarantee that execution remains safe under the current runtime context.

Low consequence does not make weak evidence trustworthy.

Vortex therefore separates these concerns and evaluates them before execution.

The architecture is intended to support explicit runtime outcomes such as:

- ALLOW;
- REVIEW;
- DENY.

Vortex does not require a single aggregate trust or risk score to represent the full runtime decision.

2. Problem Statement

Autonomous systems increasingly operate directly against deterministic infrastructure:

- filesystems;
- shells;
- APIs;
- credentials;
- CI/CD systems;
- cloud resources;
- external services.

A system may correctly identify an actor, verify its credentials, confirm its assigned permissions, and still reach an unsafe execution outcome.

The problem is that several distinct security questions are often collapsed into one:

- Is the evidence reliable?
- Is the actor authorized?
- Is the requested action within scope?
- Is the current context consistent with the authorization?
- What happens if the decision is wrong?
- Can the resulting effect be reversed?

Traditional authorization can answer whether an actor possesses permission.

Detection can describe what the actor is doing.

Neither property alone determines whether the action should execute under the current runtime conditions.

This becomes especially important for autonomous and agentic systems because context can change between actions.

Memory may change.

Tool output may change.

External state may change.

Delegated authority may change.

The sequence of individually acceptable actions may produce a larger effect than any individual action implies.

Vortex addresses this gap by placing an explicit runtime decision boundary before execution.

The decision boundary evaluates Evidence, Authority, and Consequence independently and applies policy to the combined state.

The central security question is therefore not only:

> Is this action authorized?

It is:

> Given the current evidence, authority, and consequence, should this action be allowed to execute now?

3. Design Principles

Vortex follows a small set of architectural principles that constrain how runtime decisions are made.

3.1 Separate Evidence from Authority

Evidence may increase or decrease confidence in a decision, but it must never create execution authority.

An actor with strong behavioral coherence, trustworthy telemetry, or high-confidence evidence is not automatically authorized to perform an action.

Authority must originate from an explicit authorization mechanism.

3.2 Scope Authority to the Requested Action

Authorization must be evaluated against the action that is actually being requested.

Authority should be constrained by relevant dimensions such as:

- principal;
- action;
- resource;
- environment;
- temporal bounds;
- delegation;
- approval requirements.

Possessing authority for one operation must not imply authority for another.

3.3 Fail Closed on Missing Authority

Missing, expired, ambiguous, or unverifiable authority must not be interpreted as permission.

When Vortex cannot establish the authority required for an action, execution must fail closed. Policy may return `DENY` or require explicit `REVIEW`, but unresolved authority must never permit execution.

3.4 Preserve Unknown State

Unknown values must remain unknown.

Vortex must not silently convert missing evidence, unresolved context, or unavailable authorization state into trusted defaults.

Policy may explicitly define how unknown state is handled, but the architecture must preserve the distinction between:

- known;
- unknown;
- invalid;
- denied.

3.5 Keep Consequence Independent

The potential consequence of an action must remain distinct from confidence in the evidence supporting that action.

High-confidence evidence does not reduce blast radius.

Low-risk actions do not make weak evidence more trustworthy.

Consequence informs the level of assurance, approval, or restriction that policy may require.

3.6 Re-evaluate at the Execution Boundary

Authorization established earlier in a workflow is not sufficient by itself to guarantee that execution remains appropriate later.

Relevant runtime state may change between authorization and execution.

Vortex therefore evaluates policy at the boundary where an action is about to produce an external effect.

3.7 Treat Context as Evidence, Not Authority

Runtime context may inform a policy decision, but context must not manufacture permissions.

Memory, tool output, retrieved content, prior agent reasoning, and external observations may be incomplete, stale, or adversarially influenced.

Where context affects a runtime decision, its origin and reliability should remain observable to policy.

3.8 Prefer Explicit Decisions over Implicit Trust

Vortex produces explicit runtime decisions.

The canonical decision states are:

- `ALLOW`;
- `REVIEW`;
- `DENY`.

A decision should be attributable to identifiable policy inputs rather than inferred from an opaque aggregate notion of trust.

3.9 Make Decisions Explainable

A runtime decision must be explainable from the inputs that produced it.

At minimum, an implementation should be able to identify:

- the requested action;
- the principal;
- the relevant authority;
- the evidence considered;
- the consequence characteristics;
- the policy applied;
- the resulting decision.

Explainability is part of the security model because runtime enforcement without decision provenance is difficult to audit, test, or challenge.

3.10 Authorization Is Necessary but Not Sufficient

A valid authorization establishes that an actor may perform an action within a defined scope.

It does not establish that executing the action remains appropriate under every runtime condition.

Vortex therefore distinguishes:

> authorized to act

from:

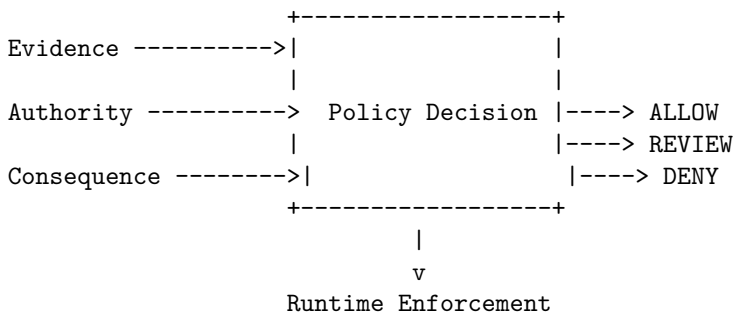
> allowed to execute now

4. Architectural Model

Vortex evaluates runtime actions through three independent security planes: Evidence, Authority, and Consequence.

These planes are intentionally not collapsed into a single trust or risk score. Each represents a different security question and retains its own semantics throughout the decision process.

Conceptually:



The policy decision is therefore a function of independent inputs:

```
Decision = Policy(Evidence, Authority, Consequence, Runtime Context)
```

This notation does not imply that the inputs are numerically aggregated.

Policy evaluates relationships and constraints between the planes while preserving their individual meaning.

In particular:

- evidence cannot create missing authority;
- authority cannot increase evidence quality;
- low consequence cannot repair insufficient authority;
- high evidence confidence cannot erase consequence;
- runtime context may constrain a previously valid authorization;
- an `ALLOW` decision applies only to the action and context that were evaluated.

The output of policy evaluation is an explicit runtime decision rather than a generalized trust score.

4.1 Evidence Plane

The Evidence Plane represents the information used to support a runtime decision.

Evidence answers:

> What supports this decision, where did it come from, and how reliable is it?

Evidence may include:

- telemetry;
- runtime observations;
- tool outputs;
- retrieved context;
- memory;
- policy inputs;
- external signals;
- prior execution results;
- integrity and provenance metadata.

Evidence is descriptive.

It may increase or decrease confidence in a runtime decision, but it does not grant execution authority.

An implementation should preserve relevant evidence properties such as:

- origin;
- provenance;
- integrity;
- freshness;
- completeness;
- consistency;
- behavioral coherence;
- transformation history.

Evidence may be:

- trusted;
- partially trusted;
- degraded;
- stale;
- incomplete;
- conflicting;
- unknown;
- adversarially influenced.

The architecture must preserve these distinctions rather than collapsing them into an implicit trusted default.

Where evidence is derived from mutable or agent-controlled sources, policy should be able to reason about its provenance.

Examples include:

- agent memory;
- retrieved documents;

- tool responses;
- intermediate model output;
- external API responses;
- chained agent messages.

The Evidence Plane therefore distinguishes:

> what the system observed

from:

> what the system can establish about the origin and reliability of that observation

Evidence may constrain a runtime decision.

Evidence must never manufacture missing authority.

4.2 Authority Plane

The Authority Plane represents the permissions that constrain what a principal may do.

Authority answers:

> Is this principal permitted to perform this specific action on this specific resource under the current constraints?

Authority may be established by mechanisms such as:

- scoped authorization;
- delegated authority;
- capability systems;
- policy bindings;
- approval workflows;
- external identity and authorization systems.

Vortex does not require authority to originate from any single authorization technology.

The architecture only requires that authority remain explicit and independently verifiable from evidence.

Relevant authority dimensions may include:

- principal identity;
- delegated actor;
- action;
- resource;
- environment;
- tenant;
- temporal bounds;
- usage limits;

- delegation chain;
- approval requirements;
- revocation state.

Authority must be evaluated against the action that is actually being requested.

A valid authorization for one action, resource, environment, or time window must not silently authorize another.

Authority may be:

- valid;
- expired;
- revoked;
- exhausted;
- out of scope;
- unverifiable;
- missing.

Missing or unverifiable authority must fail closed unless an explicit policy requires human review instead.

The Authority Plane therefore distinguishes:

> evidence that an actor appears trustworthy

from:

> proof that the actor is permitted to perform the requested action

Behavioral coherence, high-confidence telemetry, successful prior actions, or model confidence do not grant permissions.

Where external authorization systems are used, Vortex may consume their result as an Authority Plane input without redefining how that authority is established.

A valid authorization is necessary for execution.

It is not sufficient, by itself, to produce an `ALLOW` decision.

4.3 Consequence Plane

The Consequence Plane represents the potential effect of allowing an action to execute.

Consequence answers:

> If this decision is wrong, what can this action change, damage, expose, or make irreversible?

Consequence is evaluated independently from evidence confidence and execution authority.

A strongly supported and fully authorized action may still produce a high-consequence outcome.

Relevant consequence properties may include:

- reversibility;
- blast radius;
- data sensitivity;
- destructive capability;
- financial impact;
- privilege impact;
- external side effects;
- persistence;
- propagation potential;
- third-party impact.

Consequence may depend on both the individual action and its surrounding execution chain.

An action that appears reversible in isolation may contribute to an irreversible sequence when composed with other actions.

Policy should therefore be able to reason about properties such as:

- whether the effect is local or externally visible;
- whether rollback is technically possible;
- whether rollback can restore all affected parties;
- whether the action crosses a trust or administrative boundary;
- whether the action can create persistent or delegated effects;
- whether the action can amplify subsequent authority or execution capability.

The Consequence Plane therefore distinguishes:

> whether an action is permitted

from:

> what happens if permitting it turns out to be wrong

Higher consequence may require stronger evidence, narrower authority, explicit approval, or a `REVIEW` decision.

Lower consequence does not repair missing authority.

Consequence must not be used to reinterpret weak evidence as trustworthy.

Where consequence cannot be established with sufficient confidence, policy should preserve that uncertainty rather than silently assuming low impact.

4.4 Policy Decision

The Policy Decision layer evaluates the relationship between Evidence, Authority, Consequence, and relevant runtime context.

It answers:

> Given the current state of these independent inputs, should this specific action be allowed to execute now?

Policy consumes the planes without changing their semantics.

It must not convert evidence into authority, authority into evidence confidence, or consequence into permission.

A policy may express constraints such as:

- required authority for a specific action and resource;
- minimum evidence properties for a consequence class;
- mandatory review for irreversible actions;
- denial when authority is missing, expired, revoked, or out of scope;
- review when evidence is conflicting or materially incomplete;
- denial when runtime context violates an authorization constraint;
- stronger requirements when actions cross administrative or trust boundaries.

Conceptually, policy evaluates predicates over independent state:

Decision = Policy(E, A, C, R)

where:

- E represents Evidence;
- A represents Authority;
- C represents Consequence;
- R represents relevant Runtime Context.

This notation does not define a weighted score.

An implementation may use scores inside an individual plane where appropriate, but no scalar value may implicitly manufacture a property belonging to another plane.

For example:

- high evidence confidence cannot compensate for missing authority;
- valid authority cannot compensate for evidence known to be compromised;
- low consequence cannot authorize an otherwise unauthorized action;
- successful prior execution cannot create future authority.

The canonical Vortex decision states are:

- `ALLOW` — the evaluated action satisfies the policy requirements for execution;

- `REVIEW` — execution requires an explicit external decision or additional assurance;
- `DENY` — the evaluated action must not execute.

An `ALLOW` decision is scoped to the evaluated action and runtime state.

It must not be interpreted as persistent trust in the principal or as authorization for subsequent actions.

A `REVIEW` decision must not be treated as implicit permission while review is pending.

A `DENY` decision must fail closed at the execution boundary.

Policy evaluation should produce sufficient decision evidence to explain which inputs and constraints produced the result.

4.5 Runtime Enforcement

Runtime Enforcement applies the Policy Decision before the requested action can produce its external effect.

It answers:

> Can the evaluated decision actually constrain execution?

Vortex distinguishes policy evaluation from enforcement.

A policy engine may determine that an action should be denied, but that decision provides no effective security boundary if the actor can bypass the enforcement point and invoke the target directly.

The enforcement point must therefore exist on the execution path between the action-performing system and the protected resource or capability.

Conceptually:

Agent or Autonomous System | | proposed action v Vortex Decision Boundary | | ALLOW | REVIEW | DENY v Runtime Enforcement | | permitted execution only v Protected Resource / External Effect

Runtime Enforcement must preserve the semantics of the decision:

- `ALLOW` permits only the action and scope that were evaluated;
- `REVIEW` must prevent execution until the required external decision is resolved;
- `DENY` must prevent the requested action from reaching the protected capability.

Enforcement should occur as close as practical to the point where an external effect becomes possible.

Depending on the integration, enforcement may exist at boundaries such as:

- tool invocation;
- process execution;
- filesystem access;
- API invocation;
- credential use;
- CI/CD execution;
- cloud control-plane requests;
- privileged operations;
- external communication.

The agent or model must not be responsible for voluntarily enforcing its own denial.

Where possible, enforcement should be performed by deterministic runtime infrastructure outside the reasoning component.

The enforcement layer must not reinterpret `DENY` as advisory output.

It must not silently convert `REVIEW` into `ALLOW`.

It must not broaden the action, resource, or constraints represented by an `ALLOW` decision.

If enforcement state cannot be established, the protected action should fail closed.

A decision is therefore security-relevant only when the execution path makes that decision enforceable.

4.6 Audit and Decision Evidence

Vortex preserves decision evidence sufficient to reconstruct why a runtime decision was produced.

It answers:

> What was evaluated, under which policy and runtime conditions, and why did the system produce this decision?

Decision evidence is distinct from the Evidence Plane.

The Evidence Plane contains information used to support the runtime decision.

Decision evidence records the inputs, policy evaluation, and outcome associated with that decision.

An implementation should preserve, where applicable:

- decision identifier;
- timestamp;
- principal identity;
- requested action;
- target resource;

- relevant Evidence Plane inputs or references;
- authority state and scope;
- consequence characteristics;
- relevant runtime context;
- policy identifier and version;
- resulting decision;
- decision reasons;
- enforcement outcome.

Sensitive inputs do not need to be copied into an audit record when doing so would create unnecessary exposure.

Implementations may instead preserve references, hashes, integrity metadata, or other identifiers sufficient to associate the decision with the evaluated material.

Decision records should distinguish between:

- what policy decided;
- what enforcement attempted;
- what actually executed.

This distinction is important because an `ALLOW` decision does not prove that execution succeeded, and a `DENY` decision does not prove that enforcement was effective.

Where practical, decision evidence should be integrity-protected and resistant to silent modification.

Audit state should support questions such as:

- Why was this action allowed?
- Which authority permitted it?
- Which evidence influenced the decision?
- What consequence properties were considered?
- Which policy version produced the result?
- Was the decision actually enforced?
- Did the executed action match the action that was evaluated?

Auditability must not create authority.

Historical success must not be interpreted as permission for future execution.

Decision evidence exists to support accountability, investigation, testing, and verification of the runtime control boundary.

5. Decision Flow

A runtime decision begins when an actor proposes an action that may produce an external effect.

Vortex evaluates the action before that effect is allowed to occur.

The canonical decision flow is:

1. **Receive the proposed action.** Identify the principal, requested action, target resource, and relevant runtime context.
2. **Resolve Evidence Plane inputs.** Collect or reference the evidence relevant to the decision while preserving provenance, integrity, freshness, completeness, and uncertainty.
3. **Resolve Authority Plane inputs.** Determine whether the principal holds explicit authority for the requested action and resource under the current constraints.
4. **Resolve Consequence Plane inputs.** Determine the potential effect of an incorrect authorization, including reversibility, blast radius, sensitivity, persistence, and external side effects where applicable.
5. **Evaluate policy.** Apply policy to Evidence, Authority, Consequence, and Runtime Context without collapsing their semantics into a generalized trust score.
6. **Produce an explicit decision.** The result must be one of `ALLOW`, `REVIEW`, or `DENY`.
7. **Enforce the decision.** Enforcement must occur before the protected action can produce its external effect.
8. **Record decision evidence.** Preserve sufficient information to reconstruct what was evaluated, which policy produced the result, and what enforcement outcome occurred.

The flow can be represented as:

Proposed Action | v Resolve Evidence | v Resolve Authority | v Resolve Consequence | v Evaluate Policy | +——> DENY ——> Block | +——> REVIEW —-> Hold for external decision | +——> ALLOW ——> Runtime Enforcement | v External Effect | v Record Execution Outcome

The ordering above describes the logical decision process.

Implementations may resolve independent inputs concurrently, provided that all required state is available and validated before the policy decision is enforced.

A previous `ALLOW` result must not automatically authorize a subsequent action.

Where material runtime state changes between decision and execution, the action must be re-evaluated or rejected according to policy.

The execution boundary is therefore also a validation boundary.

6. Security Properties

A conforming Vortex implementation should preserve the following security properties across the runtime decision lifecycle.

6.1 No Authority from Evidence

Evidence quality, behavioral coherence, model confidence, historical success, or telemetry confidence must not create execution authority.

If required authority is absent, stronger evidence alone must not produce `ALLOW`.

6.2 Scoped Authority

Authority must be evaluated against the specific principal, action, resource, and applicable constraints.

Authorization for one action or resource must not silently extend to another.

6.3 Fail Closed on Missing Authority

Missing, expired, revoked, exhausted, out-of-scope, or unverifiable authority must not produce `ALLOW`.

Policy may produce `REVIEW` where explicitly configured, but execution must remain blocked while review is unresolved.

6.4 Preserve Unknown State

Unknown, missing, or unverifiable inputs must remain distinguishable from trusted or validated inputs.

Absence of information must not silently become positive evidence.

6.5 Consequence Independence

Consequence must remain semantically independent from evidence and authority.

Low consequence must not create authority.

High evidence confidence must not erase consequence.

6.6 Decision Scope

An `ALLOW` decision applies only to the principal, action, resource, constraints, and runtime state that were evaluated.

A decision must not become generalized trust in the principal.

6.7 Revalidation at the Execution Boundary

Material changes to authority, evidence, consequence, or runtime context between decision and execution must trigger re-evaluation or rejection according to policy.

A stale decision must not silently authorize execution under materially different conditions.

6.8 Enforcement Independence

The actor requesting an action must not be solely responsible for enforcing the decision governing that action.

Where practical, enforcement should exist outside the reasoning component and on the path to the protected capability.

6.9 Review Is Not Permission

`REVIEW` represents an unresolved decision state.

Until the required external decision is completed, the protected action must remain blocked.

6.10 Decision and Execution Integrity

The action presented to enforcement must correspond to the action evaluated by policy.

Material modification of the principal, action, resource, arguments, authority, or relevant runtime constraints after evaluation must invalidate the decision or require re-evaluation.

6.11 Decision Explainability

A runtime decision should be attributable to identifiable inputs, policy state, and decision reasons.

An implementation should be able to distinguish policy outcome, enforcement outcome, and execution outcome.

6.12 Audit Does Not Grant Authority

Historical decisions, successful executions, audit records, or prior approvals must not implicitly authorize future actions.

Audit state is evidence about prior events, not a capability.

7. Failure Modes

Vortex is designed to make several classes of runtime security failure explicit.

7.1 Evidence-to-Authority Confusion

A system treats strong evidence, behavioral coherence, or model confidence as permission to act.

Impact:

- unauthorized actions may receive implicit approval;
- confidence may become a substitute for explicit authority.

Mitigation:

- preserve Evidence and Authority as independent planes;
- fail closed when required authority is missing.

7.2 Stale Authorization

An authorization was valid when evaluated but no longer reflects the current runtime state.

Examples include:

- expiration;
- revocation;
- changed resource scope;
- changed environment;
- changed approval state;
- changed delegation.

Mitigation:

- re-evaluate material state at the execution boundary;
- reject stale decisions.

7.3 Evidence Poisoning

Evidence used by policy has been influenced by untrusted or compromised upstream state.

Possible sources include:

- poisoned memory;
- manipulated tool output;
- compromised retrieved context;
- adversarial external responses;
- chained agent messages with weak provenance.

Impact:

- policy may repeatedly re-evaluate compromised context and produce increasingly confident but incorrect decisions.

Mitigation:

- preserve evidence origin, integrity, freshness, and provenance;
- treat context as evidence rather than authority.

7.4 Enforcement Bypass

Policy produces `DENY` or `REVIEW`, but the actor can reach the protected capability without passing through enforcement.

Impact:

- policy becomes advisory rather than security-relevant.

Mitigation:

- place enforcement on the execution path;
- prevent direct access to protected capabilities where practical.

7.5 Action Mutation After Decision

The action evaluated by policy differs materially from the action presented for execution.

Examples include changes to:

- command arguments;
- resource identifiers;
- target environment;
- payload;
- privilege level;
- external destination.

Impact:

- a valid decision may be reused for an action that was never evaluated.

Mitigation:

- bind decisions to the evaluated action and relevant constraints;
- invalidate or re-evaluate modified actions.

7.6 Review Fail-Open

A `REVIEW` decision is treated as temporary permission while approval is pending or unavailable.

Impact:

- unresolved decisions may execute without the required external assurance.

Mitigation:

- treat `REVIEW` as non-executable until explicitly resolved.

7.7 Unknown-to-Trusted Conversion

Missing, unresolved, or unavailable state is silently mapped to a trusted default.

Impact:

- incomplete evidence or missing authorization state may produce false confidence.

Mitigation:

- preserve unknown state explicitly;
- require policy to define how unknown values are handled.

7.8 Consequence Collapse

Consequence is inferred from evidence confidence or authorization state rather than evaluated independently.

Impact:

- highly authorized actions may be incorrectly treated as low impact;
- strong evidence may reduce perceived blast radius without justification.

Mitigation:

- preserve consequence as an independent security plane.

7.9 Historical Trust Accumulation

Successful prior executions, audit history, or previous approvals are treated as persistent authority.

Impact:

- past behavior becomes implicit permission for future actions.

Mitigation:

- treat historical state as evidence only;
- require current authority for each decision.

7.10 Decision / Enforcement / Execution Confusion

A system records a policy decision and assumes that the corresponding enforcement or execution outcome occurred.

Examples:

- `DENY` was recorded but enforcement failed;
- `ALLOW` was recorded but execution never occurred;
- execution occurred with parameters different from the evaluated action.

Mitigation:

- record policy outcome, enforcement outcome, and execution outcome separately.

7.11 Chained Consequence Amplification

A sequence of individually acceptable actions produces a larger or irreversible effect.

Examples include:

- reversible local changes that become externally irreversible;
- delegated actions that expand future execution capability;
- multiple low-impact actions that collectively cross a trust boundary.

Mitigation:

- evaluate consequence in the context of action chains where required;
- preserve sequence-aware policy state.

7.12 Authorization Scope Creep

An authority grant intended for one action, resource, environment, or time window is interpreted more broadly.

Impact:

- permissions expand beyond their intended boundary.

Mitigation:

- evaluate explicit scope at runtime;
- reject actions outside the granted constraints.

8. Historical Provenance

The Vortex Runtime Decision Architecture evolved from earlier Detection Fidelity Score (DFS) research and subsequent runtime authorization experiments.

This section records that lineage to distinguish architectural evolution from concepts introduced only in the current specification.

8.1 Detection Fidelity Score

The earliest DFS work focused on detection fidelity under changes to the evidence pipeline.

The original problem was whether detection capability survives transformations such as redaction, anonymization, pseudonymization, telemetry loss, or other changes to the information available to detection logic.

This work established several ideas that remain relevant to Vortex:

- evidence quality can degrade;

- degradation should be measurable rather than assumed;
- missing, altered, and decayed signals have different meanings;
- detection behavior should be evaluated against explicit system conditions;
- operational trust should not depend on unexamined assumptions about telemetry.

The original DFS model was primarily concerned with detection survivability and evidence degradation rather than general-purpose runtime authorization.

8.2 Trust Decision Boundary

DFS later evolved from measuring detection survivability toward reasoning about the decisions supported by detection evidence.

A Trust Decision Boundary connected signal degradation with the action or decision that a detection could support.

This introduced an important shift:

> the security significance of evidence depends partly on what decision is being made from it

The model therefore moved beyond asking whether a detection still fires and toward asking whether the surviving evidence remains sufficient for a particular decision boundary.

This stage established the conceptual connection between evidence quality and action-sensitive decision making.

It did not establish that evidence itself grants authority.

8.3 Runtime Authorization Experiments

Subsequent DFS experiments extended the model into runtime controls for autonomous and automated systems.

These experiments introduced or explored primitives including:

- pre-execution evaluation;
- explicit agent authorization;
- scoped resources and actions;
- temporal constraints;
- single-use authorization;
- approval state;
- reversibility;
- blast radius;
- runtime circuit breaking;
- audit chaining.

These experiments demonstrated that runtime decisions require more than a measure of evidence quality.

They also exposed semantic pressure on the scalar scoring model.

Evidence confidence, behavioral coherence, action risk, reversibility, approval state, and execution authority had begun to influence the same decision path despite representing different security questions.

8.4 Separation of Evidence, Authority, and Consequence

The current Vortex architecture formalizes the resulting separation.

Rather than treating evidence quality, execution authority, and potential impact as interchangeable contributors to a generalized trust score, Vortex models them as independent security planes:

- Evidence describes what supports a decision and the reliability of that support.
- Authority describes what the principal is permitted to do.
- Consequence describes what may happen if the permitted action is wrong.

Policy evaluates relationships between these planes without allowing one plane to manufacture another.

This separation preserves useful primitives from the earlier DFS experiments while removing the semantic ambiguity created when distinct security properties were compressed into scalar decision logic.

The resulting architectural lineage is therefore:

Detection Fidelity | v Detection Survivability | v Signal Degradation | v Trust Decision Boundary | v Runtime Authorization Experiments | v Evidence / Authority / Consequence Separation | v Vortex Runtime Decision Architecture

This lineage describes architectural evolution.

It does not imply that every historical DFS implementation already satisfied the security properties defined by this specification.

9. Relationship to External Authority Systems

Vortex does not require a specific identity, authorization, or policy technology.

Authority may originate from external systems that establish what a principal is permitted to do.

Examples may include:

- identity and access management systems;
- authorization services;
- capability-based systems;
- delegated credentials;
- workload identity;
- approval systems;

- policy decision points;
- platform-native access controls.

Vortex does not treat integration with such a system as proof that execution should automatically proceed.

External authority is an input to the Authority Plane.

The runtime decision must still evaluate whether that authority applies to the specific principal, action, resource, and current constraints.

An external authority result may establish properties such as:

- principal identity;
- permitted action;
- permitted resource;
- delegation scope;
- expiration;
- environmental constraints;
- approval state;
- revocation state.

Vortex should preserve these constraints rather than reducing an external authorization result to a generic trusted or authorized flag.

For example, authority to read one resource must not become authority to modify it.

Authority valid in one environment must not silently transfer to another.

Authority valid at one point in time must not be assumed valid after expiration or revocation.

Vortex may therefore consume externally established authority while independently evaluating Evidence, Consequence, and Runtime Context.

Conceptually:

External Identity / Authority System | v Authority Plane | | Evidence —->| +—-> Vortex Policy Decision —-> Runtime Enforcement | Consequence ->| ˆ | Runtime Context

This separation allows Vortex to integrate with existing authorization infrastructure without redefining ownership of identity or permission semantics.

The external authority system answers:

> What is this principal authorized to do?

Vortex additionally asks:

> Given that authority, the available evidence, the potential consequence, and the current runtime context, may this specific action execute now?

Vortex must not broaden authority received from an external system.

Where external authority cannot be verified, is unavailable, or is outside its valid scope, the runtime decision must fail closed or require explicit review according to policy.

10. Non-Goals

This specification does not attempt to define:

- a universal identity system;
- a new credential format;
- a global authority issuance mechanism;
- a replacement for existing IAM or authorization infrastructure;
- a universal consequence taxonomy;
- a single global trust or risk score;
- an intent inference protocol;
- a specific policy language;
- a specific LLM or agent framework;
- a guarantee that authorized execution is free from operational failure.

Vortex defines a runtime decision architecture and enforcement boundary.

External systems may provide identity, authority, evidence, policy, or execution capabilities without transferring ownership of those semantics to Vortex.

11. Open Questions

The following areas remain intentionally open for future specification work:

- a canonical representation for Evidence, Authority, and Consequence inputs;
- consequence classification and composition across action chains;
- decision binding between policy evaluation and execution;
- re-evaluation rules for material runtime state changes;
- provenance requirements for mutable and agent-controlled evidence;
- interoperability with external authorization and policy systems;
- representation of human approval and review state;
- integrity protection for decision evidence;
- distributed enforcement across multi-agent workflows;
- conformance requirements for Vortex-compatible runtimes.

These questions do not change the core architectural separation defined by this specification.

They identify areas where additional contracts, schemas, or implementation guidance may be required.

12. Implementation Status

This document defines the Vortex Runtime Decision Architecture.

It is a draft specification and must not be interpreted as a claim that every property described here is already implemented by the current Vortex runtime.

The project contains runtime primitives and prior experimental implementations related to:

- authorization;
- scoped execution;
- pre-execution policy evaluation;
- runtime enforcement;
- audit state;
- tenant and runtime isolation;
- execution gating.

Implementation coverage should be evaluated independently against the security properties defined by this specification.

Future implementation work may therefore classify each normative property as:

- implemented;
- partially implemented;
- planned;
- not applicable.

Specification maturity and implementation maturity are separate concerns.

13. References

13.1 Vortex Architecture

- ADR-0001: Separate Evidence, Authority, and Consequence.
- Vortex project source history and runtime implementation.

13.2 Historical DFS Material

The historical provenance described in Section 8 is based on the earlier Detection Fidelity Score project history, including the evolution from detection fidelity and signal degradation toward decision-boundary and runtime authorization experiments.

Historical material is referenced as provenance and does not imply that earlier DFS versions implemented the current Vortex architecture.

13.3 External Systems and Specifications

External identity, authorization, capability, and policy systems may integrate with the Authority Plane.

Such systems remain independent from the Vortex architecture unless an explicit integration contract states otherwise.

Specific external specifications may be added to this section as interoperability requirements are defined.